

FROSTBIT: Proof-Carrying Execution Plans for Boolean Filtering over Compressed Bitmaps

William Liu

Exa

San Francisco, USA

wliu@exa.ai

ABSTRACT

Boolean filtering over Roaring-format compressed bitmaps is a core primitive of search and analytics systems. Prior work optimizes such queries dynamically: cost models select per-node algorithms and representations at evaluation time, buffers are allocated as results materialize, and pruning decisions are made while operands are consumed. This paper develops the static alternative for the read-only setting in which serving systems actually operate — immutable index segments and filters compiled once, evaluated many times. FROSTBIT compiles a boolean expression into a *proof-carrying execution plan*: a bottom-up analysis derives every operator’s output-container set together with a byte ceiling per container that is sound for every intermediate state of the fold, from which three guarantees follow at plan time rather than run time — execution is allocation-free by construction with a closed-form working-memory bound; a container-granular liveness mask prunes, before any input byte is read, every 64 Ki-value block that cannot contribute to the result of the whole expression; and each operator’s fold order is selected by comparing its proven accumulator footprint to the cache hierarchy. Representation and kernel choices are resolved by cost models over plan-derived quantities, including a fan-in-aware array-to-bitmap promotion rule that provably dominates the fixed cardinality thresholds used in deployed Roaring implementations.

Against both configurations of the standard Rust Roaring library, FROSTBIT improves every operation shape of a 2–16-way sweep by 1.2–15.5× (all but two cells against the library’s SIMD build), and evaluates a 25,000-tree filter corpus 3.4× and 2.25× faster end-to-end, with per-mechanism ablations — including rejected designs — isolating the contribution of each planning decision. FROSTBIT is deployed as the filtering substrate of a commercial web-scale search engine.

1 INTRODUCTION

Serving systems evaluate boolean filter predicates — conjunctions, disjunctions, and negations over per-attribute document sets — on the critical path of every query. The dominant physical representation is the Roaring bitmap [49, 62]: a two-level structure whose 64 Ki-value *containers* adaptively select among sorted arrays, packed bitmaps, and run encodings, and whose per-container kernels have been extensively optimized with SIMD techniques [37, 43, 52, 63, 70].

Optimizing the *expression* above the containers is a problem with a substantial history. Johnson [12] measured the large algorithm-choice space for compressed-bitmap operations; Amer-Yahia and Johnson [13] built a dynamic-programming optimizer that selects

per-node algorithms and intermediate formats for boolean expression trees over RLE-compressed bitmaps; Kaser and Lemire [51] fitted cost formulas to choose evaluation strategies for multi-operand Roaring queries; Kim et al. [64] select intersection algorithms per query from calibrated cost models in a production web search engine; and a theoretical line establishes worst-case-optimal evaluation of union-intersection expressions [20, 24]. All of these optimizers share a structural commitment: decisions are made *dynamically*, per evaluation, against operand statistics, and the execution machinery beneath them allocates and sizes intermediate results as they materialize.

This paper examines the opposite point in the design space, motivated by the regime in which filtering actually runs in production: index segments are immutable artifacts of an indexing pipeline, and a filter expression is compiled once and then evaluated against many segments. In this setting, dynamic re-decision is overhead with no adaptivity benefit — the operands are frozen — and we show that the entire optimization surface can be discharged *statically*, yielding plans that carry guarantees no dynamic optimizer provides:

- **Allocation-freeness as a plan-time invariant.** The analysis derives, for every operator, its exact output-container key set and a per-container byte ceiling (*capacity clamp*) sound for every intermediate state of the fold (§4.2). Consequently the execution path contains no allocation or growth code, working memory is bounded in closed form (§4.3), and results serialize in place inside the working arena (§4.4). Engines such as Trill [48] and Photon [73] pool and bound allocations operationally; CRoaring pre-sizes individual operations by cardinality analysis [49]. FROSTBIT differs in deriving the bound for the *whole expression* from the plan, and in the stronger consequence that the steady-state allocation count is exactly zero, a property we verify empirically.
- **Static, expression-wide, container-granular pruning.** An upward pass computes each node’s surviving key set; when the root’s set is narrower than a child’s, the plan carries a liveness mask and every leaf cursor skips dead containers before they are read (§7.1). Zone maps prune scans against per-block metadata [10, 56]; skip pointers and block-max structures prune *dynamically* during list traversal [7, 36]; the Java Roaring library’s workShyAnd computes a key mask at run time for flat *n*-ary intersection [79]; partition-level prefilters prune pairwise [70, 76]. FROSTBIT generalizes these to a *static* mask over arbitrary AND/OR/DIFF trees, derived from the plan itself, with a

soundness argument (Lemma 1) and with no per-evaluation decision cost.

- **Fold order selected from a proven footprint.** n -ary folds admit two orders — complete each key across all operands, or stream each operand across all keys — which are the container-algebra analogues of document-at-a-time and term-at-a-time evaluation [6], and which we show are memory-optimal in disjoint regimes on modern hierarchies. Cache-conscious cost models for operator layout are classical [3, 17, 19]; FROSTBIT’s distinction is that the discriminating quantity — the accumulator footprint — is not an estimate but the sum of proven clamps, so the decision inherits the plan’s soundness (§6).

Beneath the plan, representation and kernel selection are governed by explicit cost models over plan-derived quantities (§5); the kernels themselves synthesize the published state of the art [37, 43, 52, 72] and are verified at parity with it, so that the evaluation isolates the value of the planning layer rather than kernel differences.

Contributions. **C1.** A compilation scheme from boolean container-algebra expressions to proof-carrying plans: sound output shapes, per-container capacity clamps with a case-analytic soundness argument, and a closed-form working-memory bound; allocation-free execution and in-place result serialization follow as invariants (§4, §8). **C2.** A static, container-granular liveness analysis for arbitrary AND/OR/DIFF trees with a soundness proof and a streamed-bytes account, automatically enabled when the plan proves it profitable (§7.1). **C3.** Plan-time algorithm selection with derived break-evens: a fan-in-aware array-to-bitmap promotion rule that dominates fixed cardinality thresholds, and footprint-gated fold-order selection, including the control experiments that scoped where each applies (§5, §6). **C4.** An evaluation against both configurations of the Rust Roaring library with complete per-mechanism ablations including negative results, a 25,000-tree corpus, and a per-tree audit methodology; FROSTBIT is deployed in production and released as open source (§9, §10).

2 RELATED WORK

Expression-level optimization of bitmap and set queries. The closest line to this work begins with performance studies of compressed-bitmap operation choices [12] and the Amer-Yahia–Johnson optimizer [13], which plans boolean expression trees over RLE bitmaps by dynamic programming over per-node algorithm and format choices under density-based cost models. Kaser and Lemire [51] extend cost-based strategy selection to multi-operand Roaring queries (including heuristics for intermediate materialization), and Kim et al. [64] deploy calibrated cost models that pick intersection algorithms per query in web search. Adaptive execution takes the dynamic commitment further: Raman et al. [25] interleave RID-list intersection strategies based on observed selectivities precisely because static plans mispredict. The theory community established worst-case-optimal evaluation of union-intersection expressions [20, 24], and semijoin-style liveness reasoning over query plans dates to Yannakakis [1], recently revived as predicate transfer [77]. FROSTBIT occupies a point none of these target: in the frozen-segment regime it moves *all* decisions to compile time and, beyond selecting algorithms, emits machine-checkable guarantees — capacity

clamps, allocation-freeness, liveness masks — that dynamic optimizers by construction cannot promise before execution. The trade is explicit: FROSTBIT forgoes runtime adaptivity on interior results (§9.5) in exchange for zero per-evaluation decision cost and static safety.

Compressed bitmap structures. The word-aligned RLE family (BBC [4], WAH [21], PLWAH [29], CONCISE [28]) optimizes space and bulk boolean throughput; sorting amplifies run lengths [30]. Roaring [49, 50, 62] introduced the two-level container design FROSTBIT adopts and remains wire-compatible with. Index-level designs — variant indexes [8], bitmap join indices [5], bit-sliced arithmetic [16], design-space studies [9] — and updatable structures (UpBit [53], TEB [69, 74], CUBIT [75]) are complementary: FROSTBIT deliberately targets immutable segments, which is the assumption that makes its static analysis sound. The bitmap-versus-inverted-list boundary is mapped in [59].

Container kernels. Adaptive and instance-optimal set intersection descends from Demaine et al. [14] and the Barbay line [23, 31], with engineering treatments for inverted indexes [32, 35]. The SIMD lineage comprises shuffle-based compare-all-pairs [37, 52], branch-reduced merge networks [43], QFilter [63], FESIA’s segment prefilters [70], hierarchical partitioning (HERO [76]), and VP2INTERSECT alternatives [72]; worst-case-optimal join systems embed the same kernels relationally [45, 55]. FROSTBIT’s kernels synthesize this line; CRoaring additionally ships per-pair dispatch heuristics (galloping thresholds, cardinality-based preallocation, deferred aggregation) [49, 62] that the Rust port lacks, a distinction our evaluation accounts for (§9.5).

Block skipping and scan pruning. Zone maps and small materialized aggregates prune blocks against predicate metadata [10, 56]; self-indexing inverted files skip within lists [7]; block-max indexes prune score-bounded blocks dynamically [36]. These techniques prune either along a single scan or during traversal. FROSTBIT’s liveness analysis differs in operating *across* an arbitrary boolean expression statically, at Roaring’s own container granularity, before any operand byte is read.

Memory discipline in query engines. Streaming and vectorized engines bound steady-state allocation operationally [19, 48, 73], and cache-aware cost modeling of operator behavior is classical [3, 17]. FROSTBIT’s contribution here is the coupling: the memory bound is an artifact of the same analysis that proves capacity soundness, and serialization reuses the working arena (§4.4), which we have not seen documented for bitmap query execution.

Bitmaps in deployed systems. Signature and bitmap machinery appears throughout serving systems — signature files [2, 11], FastBit [26], BitFunnel [60], analytics engines [33, 42, 58, 65, 67, 78], graph search [41], and bit-parallel scan accelerators [39, 40, 46, 57, 66]. These document container structures and scan layouts; the expression-compilation layer described here is not, to our knowledge, described in any of them.

3 PRELIMINARIES

A FROSTBIT bitmap represents $S \subseteq [0, 2^{32})$, partitioned by key $\kappa(x) = \lfloor x/2^{16} \rfloor$ into containers over $[0, 2^{16})$, each stored as a sorted u16 array ($|c| \leq A_{\max} = 4096$), a packed bitmap of $B = 8192$ bytes,

or a run list of $\rho(c) \leq \rho_{\max} = 2047$ intervals:

$$\text{stored}(c) = \begin{cases} 2|c| & \text{array,} \\ B & \text{bitmap,} \\ 2 + 4\rho(c) & \text{runs.} \end{cases}$$

The serialized form is a 16-byte header, a structure-of-arrays directory (8 bytes per container), and 64-byte-aligned payloads; buffers are validated once at the trust boundary and read zero-copy thereafter, including from memory maps.

A filter expression is a tree over frozen-bitmap leaves with operators $\wedge$, $\vee$ (arbitrary arity), and $\setminus$ (binary). Same-operator edges are flattened at construction; an operator's *fan-in* n is its flattened operand count. $K(v)$ denotes the key set node v can emit; hats $\widehat{}$ denote plan-time upper bounds.

4 PLAN COMPILATION

Expression construction and analysis are fused: each combinator splices its children's programs into a flat post-order instruction sequence and computes the artifacts below bottom-up. A plan is constructed once and evaluated arbitrarily many times.

4.1 Shapes

DEFINITION 1 (SHAPE). *The shape of node v is the key-sorted sequence $\{(k, \widehat{\text{card}}_k, \widehat{\rho}_k, \widehat{\tau}_k) : k \in K(v)\}$ of per-key bounds on output cardinality, run count, and representation under the kernel decision ladder.*

Leaf shapes are exact. Interior shapes follow operator semantics with saturating interval arithmetic ($\cap/\min$ for $\wedge$; $\cup/\Sigma$ for $\vee$; left projection for $\setminus$) and are sound over-approximations of any execution. Because flattening changes arity, each spliced child contributes its *operand weight* to the parent's per-key fan-in bound $\widehat{n}_k$; weights make $\widehat{n}_k$ an exact upper bound on executor fan-in, which the promotion rule of §5 requires. (An early implementation that bounded fan-in by tree arity rather than flattened weight produced plans inconsistent with their kernels; the capacity stress suite of §4.2 detects this class of error, and the final design eliminates it by making the plan's clamp the sole authority for representation choice.)

4.2 Capacity clamps

Each operator receives an arena plan: one slot per key of its shape, with capacity

$$\text{cap}(k) = \begin{cases} \sigma(\widehat{\text{card}}_k) & \wedge \\ \max(\text{stored}_{\text{lhs}}(k), \sigma(\widehat{\text{card}}_k)) & \setminus \\ \max(2\widehat{\text{card}}_k, 2+4\widehat{\rho}_k, \max_i \text{stored}_i(k)) \text{ or } B & \vee \end{cases}$$

where $\sigma(m) = 2m$ for $m \leq A_{\max}$ and B otherwise, and the union clamp is B exactly when the plan assigns the key a bitmap or native-run accumulator (§5).

PROPOSITION 1 (CAPACITY SOUNDNESS). *For every operator, key, and input consistent with the shapes, every intermediate state of the fold at key k occupies at most $\text{cap}(k)$ bytes.*

ARGUMENT. By case analysis on the accumulator ladder. For $\wedge$, the accumulator seeds from the per-key minimum-cardinality operand and only shrinks; all reachable states (array, run, bitmap)

fit $\sigma(\widehat{\text{card}}_k)$. For $\setminus$, untouched keys are verbatim copies; touched keys shrink from the left operand, and run-splitting states, bounded by $2 + 4(\rho_{\text{lhs}} + \text{items}_{\text{rhs}})$ bytes, arise only under a B clamp with conversion to bitmap forced before $\rho_{\max}$ is exceeded. For $\vee$, array accumulation is planned only when the summed bound fits an array, in which case every prefix fits; otherwise the clamp is B , which dominates bitmap, run, and conversion states. The executor never re-derives these decisions — the clamp is the decision — so plan/kernel divergence cannot occur. $\square$

We state Proposition 1 and its relatives formally because the memory bound and the executor's structure depend on them, not as contributions to theory: the arguments are elementary. Their value is operational — debug builds assert every recorded container against its clamp, and an 11,600-case randomized stress suite exercises the obligations.

INVARIANT 1 (ALLOCATION-FREENESS). *The execution path contains no allocation, growth, or reallocation code.*

4.3 Working-memory bound

An operator with n_s slots occupies one arena comprising a reserved directory region $H(n_s) = \text{align}_{64}(16 + 8n_s)$, the slot region, an optional mirror region for side-flipped folds (§6), and fixed scratch $2B$:

$$A(P) = H(n_s) + (1 + \delta_P) \text{align}_{64}\left(\sum_k \text{align}_8(\text{cap}(k))\right) + 2B. \quad (1)$$

Since $\text{cap} \leq B$, $A(P) \leq H(n_s) + (1 + \delta_P)n_s B + 2B$, and $\delta_P = 1$ only when $\sum_k \text{cap}(k) \leq F_{\max} = 64 \text{ KiB}$, so the mirror contributes at most 64 KiB. For a tree, arenas are live along the evaluation stack, and peak working memory is the plan-time quantity $\max_s \sum_{v \in s} A(P_v)$ over stack states s .

4.4 In-place serialization

Because the directory region is reserved and slot capacities dominate payloads, serialization compacts payloads leftward within the arena and emits the arena's own buffer as the result.

PROPOSITION 2 (LEFTWARD COMPACTION). *Writing outputs in key order at $d_i = \text{base} + \sum_{j < i} \text{size}_j$ (with per-type alignment) satisfies $d_i \leq s_i$ for every source offset s_i ; no unread byte is overwritten.*

ARGUMENT. $s_i = H + \sum_{j < i} \text{align}_8(\text{cap}_j) + \Delta_i \geq \text{base} + \sum_{j < i} \text{size}_j = d_i$, as $\text{base} \leq H$ and $\text{size}_j \leq \text{cap}_j$ pointwise; mirror-side sources lie beyond the primary region, and alignment rounding preserves the inequality because sources are at least as aligned as destinations. $\square$

The operational content is one full pass of memory traffic removed relative to engines that copy results out of working memory. Profiling during development isolated this term directly: FROSTBIT's 16-way difference fold was already faster than the baseline (185 vs. 190 ns) while the end-to-end operation was slower (206 vs. 190 ns); the difference was the output copy, and eliminating it reversed every large-result cell (§9.3).

Table 1: Calibrated kernel constants (Apple M-series; medians over criterion runs).

c_b	per 8-lane merge block (compare-all-pairs)	1.45 ns
c_m	per element streamed through an array merge	0.85 ns
c_s	per value scattered into a bitmap	1.2 ns
c_0	bitmap clear + population count (fixed)	450 ns
c_p	bitmap membership probe (hoisted loop)	1.0 ns
c_g	galloping search probe (incl. mispredictions)	2.5 ns

5 REPRESENTATION AND KERNEL SELECTION

Execution decisions are functions of plan-derived quantities with constants calibrated per microarchitecture (Table 1).

Sorted-array kernel tiers. For an array pair with $|A| \leq |B|$: balanced pairs ($|B| \leq 4|A|$) run a shuffle-merge (compare-all-pairs over rotations of both operands — five permutations per 8-lane block rather than seven — with table-driven output compaction) at cost $c_b \lceil (|A|+|B|)/8 \rceil$; heavily skewed pairs run galloping binary search at $\approx c_g |A| \log_2 |B|$, admitted only under the margin test $8 |A| \log_2 |B| < |B|$ so that the branch-heavy search must dominate the middle tier’s linear window advance by a factor, not merely match it; intermediate skews run a galloping broadcast scan. Both boundaries were set empirically; replacing the margin test with a fixed ratio regressed five-way conjunctions by 26%.

Fan-in-aware promotion. A union key accumulating in array form re-streams its accumulator at every merge; with fan-in n and summed cardinality s ,

$$C_{\text{arr}}(n, s) \approx c_m s \left(\frac{n+1}{2} - \frac{1}{n} \right), \quad C_{\text{bmp}}(s) \approx c_0 + c_s s, \quad (2)$$

whose crossover yields no promotion at $n \leq 3$ for any $s \leq A_{\text{max}}$ and thresholds $s \gtrsim 980, 480, 200$ at $n = 4, 5, 8$, encoded as $\text{PROMOTE}(n, s) \equiv \text{so no contributing container is pruned}$. Deployed Roaring implementations promote at a fixed cardinality (256 in CRoaring’s deferred aggregation [49, 62]); the fixed rule is dominated in both directions, and we measured both failures: tuned low it converts the OR-groups of low-fan-in conjunctions into bitmaps their parent consumes more slowly (+29% on that shape); tuned high it strands wide unions in array form (3.7 $\times$ on 4-way unions; Table 4). Fan-in bounds come from the weighted shapes of §4.1.

Run containers fold natively (run $\times$ run intersection, union, and run-splitting difference on typed interval slices) inside B-clamped slots, converting to bitmaps only on proven overflow.

6 FOLD-ORDER SELECTION

An n -ary fold traverses keys $\times$ operands in one of two orders. *Key-major* order completes each key before the next: one accumulator remains cache-resident, but each operand stream is resumed once per key — nK prefetch restarts over K keys. *Partner-major* order streams each operand end-to-end against all accumulators: perfectly sequential input traffic, but the accumulator set is revisited n times. These are the container-algebra analogues of document-at-a-time and term-at-a-time postings traversal [6], and both move identical byte volumes; they differ only in arrangement relative to the cache hierarchy.

Profiling 16-way differences quantified the key-major penalty: the merge kernel sustains 1.45 ns per 8-lane block in isolation and 1.56 ns in a single-key control, but 2.17 ns inside a 32-key key-major fold (+50%), attributable to resuming 16 interleaved streams per key. Partner-major execution recovered the isolated rate — and dense unions then exhibited the inverse pathology: with per-key bitmap accumulators the accumulator set is $K \cdot B$ (128 KiB at $K=16$), revisiting it each pass displaces L1, and key-major won by 12%.

The plan resolves the choice without estimation: the accumulator footprint $F(P) = \sum_k \text{cap}(k)$ is already proven (Eq. 1). FROST-BIT folds partner-major iff $F(P) \leq F_{\text{max}} = 64 \text{ KiB}$ (half of L1D, reserving room for one streaming operand and the mirror side), in which case the plan allocates the mirror region and array merges alternate slot sides, eliminating staging copies. Intersection is the measured exception: a key-count scaling control (16-way intersection over 1 vs. 32 keys: 695 ns $\rightarrow$ 21.6 ns, i.e. 31.1 $\times$ for 32 $\times$ the keys) shows no super-linear term — the accumulator collapses after the first operand and subsequent operands are searched, not streamed — and partner-major seeding would require clamping by the driver’s cardinality rather than the per-key minimum, weakening Proposition 1. Intersection therefore remains key-major by measurement, difference and union dispatch on $F(P)$.

7 LIVENESS ANALYSIS

7.1 Static container pruning

DEFINITION 2 (LIVENESS MASK). $\mathcal{M}(T) = K(\text{root}(T))$ under the shape recurrences: intersection of child key sets at $\wedge$, union at $\vee$, left child at $\setminus$.

LEMMA 1 (PRUNING SOUNDNESS). For any leaf ℓ and key $k \notin \mathcal{M}(T)$, deleting container $c_k(\ell)$ from the input changes no output of T . Moreover $\mathcal{M}(T)$ over-approximates the true surviving key set, so no contributing container is pruned.

ARGUMENT. Induction on T : a key absent from $K(v)$ yields the empty container at v ; empty containers are identities for $\vee$, annihilators for $\wedge$, and left-absorbing for $\setminus$, so absence propagates to the root. Executor key sets are subsets of shape key sets (§4.1), giving the over-approximation. $\square$

The planner materializes $\mathcal{M}(T)$ as an 8 KiB bitset whenever $|K(\text{root})| < \max_i |K(v_i)|$ — exactly the condition under which pruning is non-vacuous — and every leaf cursor skips dead keys in place. The streamed-bytes reduction is

$$\Delta = \sum_{\ell \in \text{leaves}(T)} \sum_{k \in K(\ell) \setminus \mathcal{M}(T)} \text{stored}(c_k(\ell)), \quad (3)$$

which for the selective-filter benchmark of §9 (one 4-key conjunct against four 256-key disjunction legs) removes 98% of input bytes and yields 7.35 ns versus 241 ns unpruned; enabling the automatic derivation improved the full 25k-tree corpus by 15.3% ($p < 0.05$). Relative to runtime masking for flat intersections [79] and scan-side zone-map pruning [10, 56], the analysis here is static, applies to arbitrary operator trees, and carries a soundness argument tied to the plan’s shapes.

7.2 Compiled short-circuits

The instruction sequence carries *guards* after each non-flattened $\wedge$ -operand and each $\setminus$ -minuend: an empty operand pops the partial state and branches past the fold, so sibling subtrees are never evaluated. Guards use relative offsets and therefore survive splicing into enclosing plans. Within operators, intersection fuses population count into the word-level AND and abandons a key on emptiness; per-key folds order operands most-selective-first; partner-major passes skip dead slots. A filter containing a contradictory subexpression evaluates in 133 ns against 392 *ns* for the baseline, since the expensive sibling is never executed; we report this as a property of guard placement rather than a headline result.

8 MEMORY DISCIPLINE

Four per-thread pools back execution: op arenas (Eq. 1), fold cursor/reference scratch, the evaluator’s operand stack, and result buffers. Pooled buffers are stored empty and loaned at the caller’s lifetime; intermediate arenas chain directly as operator inputs, and in-place serialization (Proposition 2) swaps the arena’s buffer out as the result while a pooled result buffer rotates back. Steady-state evaluation therefore performs zero heap allocations — verified by sampling profiles in which the allocator does not appear — and first use on a thread allocates once, after which buffers circulate. The comparable baseline cost is visible in its own profiles: approximately 6% of the Rust Roaring library’s runtime in our workloads is allocator time, before any tail-latency effects.

9 EVALUATION

Setup. Apple M-series (aarch64/NEON); stable Rust for FROSTBIT and the default baseline, nightly for the baseline’s simd feature; criterion medians over 30 samples $\times$ 4 s per benchmark (corpus: 10×20 s), typical confidence half-widths under 2%. Baselines: Rust roaring 0.11.4 in both configurations — the default build (scalar array kernels; what stable-toolchain users run) and the nightly-only simd build — interleaved by a two-pass runner into one report. All FROSTBIT tree measurements include plan construction. Correctness: differential suites against Roaring (random trees, pruned trees, run containers, 10M-element round trips) and the 11.6k-case clamp stress; every configuration reported here passes all suites.

9.1 Operation sweep

Table 2 reports the full 2–16-way sweep over three container regimes. Against the default build FROSTBIT improves every cell by 1.2–15.5 $\times$; against the SIMD build every cell but two at 16-way, which sit within noise (0.98 $\times$, 0.99 $\times$).

9.2 Filter-tree corpus

End-to-end evaluation uses 25,000 deterministic random filter trees (295,455 leaves; log-spread sizes 2–100 leaves; depths 2–15; per-tree shape profiles spanning deep conjunction chains, flat 100-way disjunctions, and difference-heavy composites; 100 trees pinned to both extremes), each built, planned, and materialized per iteration. FROSTBIT: 2.63 s (9.5K trees/s); default baseline 9.06 s (3.45 $\times$); SIMD baseline 5.92 s (2.25 $\times$). Table 3 reports named shapes bracketing production patterns.

A per-tree audit — minimum-of-five timing of every corpus tree on both engines, structural feature extraction over trees where FROSTBIT trails, and root-cause analysis of the worst offenders — is part of the methodology: it directed two of the shipped mechanisms (the hoisted bitmap-membership loop, reducing the audit’s offender count from 1,245 to 743 of 25,000; then automatic liveness derivation, the corpus-level –15.3%).

9.3 Ablations

Every mechanism was admitted under a keep-only-if-faster rule; Table 4 reports the ledger, including rejected designs, whose negative results delimit the design space. Three observations. First, the in-place-serialization entry illustrates layer-correct profiling: the fold outperformed the baseline while the operation lost, and the entire gap was the output copy. Second, branch discipline is empirical, not doctrinal: an adversarial-versus-predictable input sweep showed the merge advance branches cost nothing measurable; a branchless rewrite of the scalar union merge *regressed* 26% (predictable branches beat unconditional work); table-driven compaction of the intersection emit won 2.2 $\times$ at dense overlap once gated to cost nothing at sparse overlap. Third, the fold-order controls (§6) scoped partner-major execution to the operators where the memory system rewards it.

9.4 Kernel parity

Because the claims concern the planning layer, we verified the kernels are not the explanation: per-block throughput against the baseline’s SIMD kernels at equal work shows parity to +16%, consistent with attributing the end-to-end differences of Tables 2–3 to planning, pruning, and memory discipline.

9.5 Limitations

Four limitations bound the present evaluation. (1) All measurements are from one aarch64 microarchitecture; the x86 (SSE) kernel ports compile and are algorithmically identical, but their performance and the transfer of Table 1’s constants are unvalidated. (2) The baseline family is the Rust Roaring library; CRoaring ships per-pair dispatch heuristics its Rust port lacks, so a CRoaring comparison with per-mechanism attribution is required before the end-to-end multiples can be read as pure planning-layer gains. (3) Operands and trees are synthetic (deterministic generators); standard Roaring real-data corpora and production filter traces remain future work. (4) Interior-node bounds are sound but loose, and static dispatch on loose bounds forgoes the adaptivity of dynamic schemes [25]; quantifying bound tightness and mis-dispatch frequency against an oracle is open. The evaluation is single-threaded; the per-thread pool design has not been characterized under concurrency.

10 DEPLOYMENT

FROSTBIT is deployed as the filtering substrate of Exa, a commercial web-scale search engine, where every boolean filter predicate compiles to a FROSTBIT expression evaluated against frozen per-segment bitmaps served from memory maps. The deployment exercises the design as intended — plans compiled once per query and

Table 2: Full operation sweep, medians in μ s: FROSTBIT / Roaring default / Roaring simd. Sparse = 32-key arrays (800/key); dense = 16-key bitmaps (5000/key); runs = 16-key run containers. FROSTBIT values reflect the complete system.

op	regime	2-way	4-way	8-way	16-way
$\wedge$	sparse	10.4 / 63.5 / 13.9	14.2 / 82.3 / 24.6	16.7 / 73.0 / 24.7	20.4 / 73.3 / 25.1
$\wedge$	dense	15.2 / 18.3 / 17.8	13.6 / 22.9 / 21.5	12.4 / 22.5 / 22.1	14.8 / 22.8 / 22.3
$\wedge$	runs	0.91 / 1.25 / 1.21	1.60 / 3.10 / 3.10	2.70 / 4.60 / 4.40	4.70 / 4.80 / 4.70
$\vee$	sparse	46.7 / 89.5 / 62.7	54.9 / 517.6 / 272.8	117.1 / 1200 / 789	227.0 / 1390 / 956.6
$\vee$	dense	5.20 / 8.20 / 8.10	9.90 / 18.3 / 18.3	20.8 / 39.0 / 38.8	42.4 / 80.0 / 79.4
$\vee$	runs	0.92 / 1.40 / 1.30	1.60 / 3.30 / 3.30	3.00 / 7.10 / 7.20	5.60 / 11.4 / 11.5
$\setminus$	sparse	11.5 / 122.3 / 12.9	31.5 / 480.5 / 35.6	71.6 / 1110 / 80.2	188.0 / 2310 / 183.7
$\setminus$	dense	5.80 / 8.00 / 8.10	48.4 / 61.0 / 62.0	53.3 / 158 / 155	65.7 / 280.6 / 277.6
$\setminus$	runs	0.75 / 0.95 / 0.94	1.20 / 2.20 / 2.20	2.00 / 5.00 / 5.10	3.60 / 9.70 / 9.90

Table 3: Filter-tree shapes, medians (μ s); plan construction included.

shape	FROSTBIT	R	R ^{simd}
conj5 (nested $\wedge$, flattens 5-way)	34.1	315	80.3
filter (base $\wedge$ OR-group $\wedge$ $\neg$ lang)	17.5	41.0	43.7
dnf ($\vee$ of $\wedge$ -groups)	51.8	582	78.6
cnf3 ($\wedge$ of $\vee$ -groups)	72.6	331	102
selective (statically pruned)	7.35	846	374
contradiction (guarded)	0.133	820	392
25k corpus (seconds)	2.63	9.06	5.92

Table 4: Ablation ledger (measured; kept above the line, rejected below).

mechanism	effect
balanced shuffle-merge (replacing scan)	dnf 130 $\rightarrow$ 57 ns
register-resident merge rewrite	2-way $\wedge$: 49 $\rightarrow$ 24.8 ns
fan-in-aware promotion	4-way $\vee$: 203 $\rightarrow$ 54.9 ns
partner-major (footprint-gated)	8-way sparse: 85 $\rightarrow$ 72 ns
in-place serialization	dense 2-way $\vee$: 7.2 $\rightarrow$ 5.1 ns
hoisted bitmap membership loop	audit offenders 1245 $\rightarrow$ 743
automatic liveness derivation	corpus 3.10 $\rightarrow$ 2.63 ns
gated branchless $\wedge$ emit	dense-hit emit 2.07 $\rightarrow$ 0.95 ns
eager bitmap intermediates ($s > 256$)	cnf3 89 $\rightarrow$ 115 ns
branchless scalar $\vee$ merge	72 $\rightarrow$ 91 ns
deferred vector-mask accumulation	$\setminus$: 83 $\rightarrow$ 97 ns
window pre-clipping before merges	redundant with galloping
partner-major $\wedge$	no gain (linear-scaling control)

replayed across segments; immutable segments produced by the indexing pipeline; allocation-free steady-state evaluation — and motivated the original design constraints, in particular the removal of allocator traffic from the tail. A quantitative production characterization (tail-latency distributions, filter-shape statistics, fleet-scale resource accounting) is beyond the scope of this paper.

11 CONCLUSION

For boolean filtering over compressed bitmaps in the frozen-segment serving regime, the optimization surface that prior work resolves dynamically can be discharged entirely at plan time, and doing so

yields guarantees dynamic schemes cannot offer: allocation-free execution under a closed-form memory bound, sound static pruning at container granularity across arbitrary expression trees, and fold orders selected from proven footprints. On kernels held at parity with the published state of the art, these plan-time guarantees account for 1.2–15.5 $\times$ per operation and 2.25–3.45 $\times$ end-to-end against the standard Rust Roaring library. The system, its differential and stress suites, the 25k-tree corpus, and the audit tooling are open source.

REFERENCES

- [1] M. Yannakakis. Algorithms for acyclic database schemes. *Vldb*, 1981.
- [2] C. Faloutsos, S. Christodoulakis. Signature files: an access method for documents and its analytical performance evaluation. *ACM TOIS* 2(4), 1984.
- [3] M. E. Wolf, M. S. Lam. A data locality optimizing algorithm. *PLDI*, 1991.
- [4] G. Antoshenkov. Byte-aligned bitmap compression. *DCC*, 1995.
- [5] P. O’Neil, G. Graefe. Multi-table joins through bitmapped join indices. *SIGMOD Record* 24(3), 1995.
- [6] H. Turtle, J. Flood. Query evaluation: strategies and optimizations. *Information Processing & Management* 31(6), 1995.
- [7] A. Moffat, J. Zobel. Self-indexing inverted files for fast text retrieval. *ACM TOIS* 14(4), 1996.
- [8] P. O’Neil, D. Quass. Improved query performance with variant indexes. *SIGMOD*, 1997.
- [9] C.-Y. Chan, Y. E. Ioannidis. Bitmap index design and evaluation. *SIGMOD*, 1998.
- [10] G. Moerkotte. Small materialized aggregates: a light weight index structure for data warehousing. *Vldb*, 1998.
- [11] J. Zobel, A. Moffat, K. Ramamohanarao. Inverted files versus signature files for text indexing. *ACM TODS* 23(4), 1998.
- [12] T. Johnson. Performance measurements of compressed bitmap indices. *Vldb*, 1999.
- [13] S. Amer-Yahia, T. Johnson. Optimizing queries on compressed bitmaps. *Vldb*, 2000.
- [14] E. D. Demaine, A. López-Ortiz, J. I. Munro. Adaptive set intersections, unions, and differences. *SODA*, 2000.
- [15] A. Moffat, L. Stuver. Binary interpolative coding for effective index compression. *Inf. Retrieval* 3(1), 2000.
- [16] D. Rinfret, P. O’Neil, E. O’Neil. Bit-sliced index arithmetic. *SIGMOD*, 2001.
- [17] S. Manegold, P. Boncz, M. Kersten. Generic database cost models for hierarchical memory systems. *Vldb*, 2002.

- [18] V. N. Anh, A. Moffat. Inverted index compression using word-aligned binary codes. *Inf. Retrieval* 8(1), 2005.
- [19] P. Boncz, M. Zukowski, N. Nes. MonetDB/X100: hyper-pipelining query execution. *CIDR*, 2005.
- [20] E. Chiniforooshan, A. Farzan, M. Mirzazadeh. Worst case optimal union-intersection expression evaluation. *ICALP*, 2005.
- [21] K. Wu, E. J. Otoo, A. Shoshani. Optimizing bitmap indices with efficient compression. *ACM TODS* 31(1), 2006.
- [22] M. Zukowski, S. Héman, N. Nes, P. Boncz. Super-scalar RAM-CPU cache compression. *ICDE*, 2006.
- [23] J. Barbay, A. López-Ortiz, T. Lu. Faster adaptive set intersections for text searching. *WEA*, 2006.
- [24] P. Bille, A. Pagh, R. Pagh. Fast evaluation of union-intersection expressions. *ISAAC*, 2007.
- [25] V. Raman, L. Qiao, W. Han, I. Narang, Y. Chen, K.-H. Yang, F.-L. Ling. Lazy, adaptive RID-list intersection, and its application to index and-ing. *SIGMOD*, 2007.
- [26] K. Wu et al. FastBit: interactively searching massive data. *J. Phys.: Conf. Ser.* 180, 2009.
- [27] H. Yan, S. Ding, T. Suel. Inverted index compression and query processing with optimized document ordering. *WWW*, 2009.
- [28] A. Colantonio, R. Di Pietro. CONCISE: compressed 'n' composible integer set. *Inf. Process. Lett.* 110(16), 2010.
- [29] F. Delière, T. B. Pedersen. Position list word aligned hybrid: optimizing space and performance for compressed bitmaps. *EDBT*, 2010.
- [30] D. Lemire, O. Kaser, K. Aouiche. Sorting improves word-aligned bitmap indexes. *Data & Knowl. Eng.* 69(1), 2010.
- [31] J. Barbay, A. López-Ortiz, T. Lu, A. Salinger. An experimental investigation of set intersection algorithms for text searching. *ACM JEA* 14, 2010.
- [32] J. S. Culpepper, A. Moffat. Efficient set intersection for inverted indexing. *ACM TOIS* 29(1), 2010.
- [33] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, T. Vassilakis. Dremel: interactive analysis of web-scale datasets. *PVLDB* 3(1), 2010.
- [34] B. Schlegel, R. Gemulla, W. Lehner. Fast integer compression using SIMD instructions. *DaMoN*, 2010.
- [35] B. Ding, A. C. König. Fast set intersection in memory. *PVLDB* 4(4), 2011.
- [36] S. Ding, T. Suel. Faster top-k document retrieval using block-max indexes. *SIGIR*, 2011.
- [37] B. Schlegel, T. Willhalm, W. Lehner. Fast sorted-set intersection using SIMD instructions. *ADMS@VLDB*, 2011.
- [38] S. Vigna. Quasi-succinct indices. *WSDM*, 2013.
- [39] Y. Li, J. M. Patel. BitWeaving: fast scans for main memory data processing. *SIGMOD*, 2013.
- [40] L. Sidiropoulos, M. Kersten. Column imprints: a secondary index structure. *SIGMOD*, 2013.
- [41] M. Curtiss et al. Unicorn: a system for searching the social graph. *PVLDB* 6(11), 2013.
- [42] F. Yang, E. Tschetter, X. Léauté, N. Ray, G. Merlino, D. Ganguli. Druid: a real-time analytical data store. *SIGMOD*, 2014.
- [43] H. Inoue, M. Ohara, K. Taura. Faster set intersection with SIMD instructions by reducing branch mispredictions. *PVLDB* 8(3), 2014.
- [44] G. Ottaviano, R. Venturini. Partitioned Elias-Fano indexes. *SIGIR*, 2014.
- [45] T. L. Veldhuizen. Leapfrog Triejoin: a simple, worst-case optimal join algorithm. *ICDT*, 2014.
- [46] Z. Feng, E. Lo, B. Kao, W. Xu. ByteSlice: pushing the envelop of main memory data processing with a new storage layout. *SIGMOD*, 2015.
- [47] D. Lemire, L. Boytsov. Decoding billions of integers per second through vectorization. *Softw. Pract. Exp.* 45(1), 2015.
- [48] B. Chandramouli, J. Goldstein, M. Barnett, R. DeLine, D. Fisher, J. C. Platt, J. F. Terwilliger, J. Wernsing. Trill: a high-performance incremental query processor for diverse analytics. *PVLDB* 8(4), 2015.
- [49] S. Chambi, D. Lemire, O. Kaser, R. Godin. Better bitmap performance with Roaring bitmaps. *Softw. Pract. Exp.* 46(5), 2016.
- [50] D. Lemire, G. Ssi-Yan-Kai, O. Kaser. Consistently faster and smaller compressed bitmaps with Roaring. *Softw. Pract. Exp.* 46(11), 2016.
- [51] O. Kaser, D. Lemire. Compressed bitmap indexes: beyond unions and intersections. *Softw. Pract. Exp.* 46(2), 2016.
- [52] D. Lemire, L. Boytsov, N. Kurz. SIMD compression and the intersection of sorted integers. *Softw. Pract. Exp.* 46(6), 2016.
- [53] M. Athanassoulis, Z. Yan, S. Idreos. UpBit: scalable in-memory updatable bitmap indexing. *SIGMOD*, 2016.
- [54] L. Dhulipala, I. Kabiljo, B. Karrer, G. Ottaviano, S. Pupyrev, A. Shalita. Compressing graphs and indexes with recursive graph bisection. *KDD*, 2016.
- [55] C. R. Aberger, A. Lamb, S. Tu, A. Nötzli, K. Olukotun, C. Ré. Empty-Headed: a relational engine for graph processing. *SIGMOD*, 2016.
- [56] B. Dageville et al. The Snowflake elastic data warehouse. *SIGMOD*, 2016.
- [57] H. Lang, T. Mühlbauer, F. Funke, P. Boncz, T. Neumann, A. Kemper. Data Blocks: hybrid OLTP and OLAP on compressed storage. *SIGMOD*, 2016.
- [58] S. Chambi, D. Lemire, R. Godin, K. Boukhalfa, C. Allen, F. Yang. Optimizing Druid with Roaring bitmaps. *IDEAS*, 2016.
- [59] J. Wang, C. Lin, Y. Papakonstantinou, S. Swanson. An experimental study of bitmap compression vs. inverted list compression. *SIGMOD*, 2017.
- [60] B. Goodwin, M. Hopcroft, D. Luu, A. Clemmer, M. Curmei, S. Elnikety, Y. He. BitFunnel: revisiting signatures for search. *SIGIR*, 2017.
- [61] G. E. Pibiri, R. Venturini. Clustered Elias-Fano indexes. *ACM TOIS* 36(1), 2017.
- [62] D. Lemire, O. Kaser, N. Kurz, L. Deri, C. O'Hara, F. Saint-Jacques, G. Ssi-Yan-Kai. Roaring bitmaps: implementation of an optimized software library. *Softw. Pract. Exp.* 48(4), 2018.
- [63] S. Han, L. Zou, J. X. Yu. Speeding up set intersections in graph algorithms using SIMD instructions. *SIGMOD*, 2018.
- [64] S. Kim, T. Lee, S.-w. Hwang, S. Elnikety. List intersection for web search: algorithms, cost models, and optimizations. *PVLDB* 12(1), 2018.
- [65] J.-F. Im et al. Pinot: realtime OLAP for 530 million users. *SIGMOD*, 2018.
- [66] B. Hentschel, M. S. Kester, S. Idreos. Column Sketches: a scan accelerator for rapid and robust predicate evaluation. *SIGMOD*, 2018.
- [67] B. Chattopadhyay et al. Procella: unifying serving and analytical data at YouTube. *PVLDB* 12(12), 2019.
- [68] J. Mackenzie, A. Mallia, M. Petri, J. S. Culpepper, T. Suel. Compressing inverted indexes with recursive graph bisection: a reproducibility study. *ECIR*, 2019.
- [69] H. Lang, A. Beischl, V. Leis, P. Boncz, T. Neumann, A. Kemper. Tree-encoded bitmaps. *SIGMOD*, 2020.
- [70] J. Zhang, Y. Lu, D. G. Spampinato, F. Franchetti. FESIA: a fast and SIMD-efficient set intersection approach on modern CPUs. *ICDE*, 2020.
- [71] G. E. Pibiri, R. Venturini. Techniques for inverted index compression. *ACM Comput. Surv.* 53(6), 2021.
- [72] G. Diez-Cañas. Faster-than-native alternatives for x86 VP2INTERSECT instructions. arXiv:2112.06342, 2021.
- [73] A. Behm et al. Photon: a fast query engine for lakehouse systems. *SIGMOD*, 2022.
- [74] M. P. Saputra, E. Tzirita Zacharitou et al. In-place updates in tree-encoded bitmaps. *DaMoN*, 2022.

- [75] J. Wang, M. Athanassoulis. CUBIT: concurrent updatable bitmap indexing. *PVLDB* 17, 2024.
- [76] B. Yang, W. Zheng, X. Lian, Y. Cai, X. S. Wang. HERO: a hierarchical set partitioning and join framework for speeding up the set intersection over graphs. *Proc. ACM Manag. Data (SIGMOD)*, 2024.
- [77] Y. Yang, H. Zhang, P. Padmanabha Iyer, X. Yu. Predicate transfer: efficient pre-filtering on multi-join queries. *CIDR*, 2024.
- [78] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, A. Milovidov. ClickHouse — lightning fast analytics for everyone. *PVLDB* 17(12), 2024.
- [79] The RoaringBitmap Java library, `org.roaringbitmap` (workShyAnd aggregation). <https://github.com/RoaringBitmap/RoaringBitmap>, accessed 2026.